

assignment 1: observability

Hamdan ul aziz 08895

project name: quickbite

repo: quickbite-observability

overview

quickbite is a small food ordering api that i built for this assignment. it is written in node.js with express. i added metrics and logging to it and then used those metrics and logs to investigate problems following parts a to e of the assignment.

the full stack of this is quickbite app (node/express) - prometheus + grafana for metrics, node exporter for host metrics and filebeat + elasticsearch + kibana for logs. everything runs with the docker compose.

part a: my project

The problem. food ordering apps (and most backend apps in general) fail in ways that are not visible until a customer complains so orders can quietly pile up in the kitchen queue and one slow dependency can drag down latency for everyone even if nothing is actually "down" and a careless debugging label (like tagging a metric by request id) can silently blow up the metrics backend. So i wanted to build something simple that shows all three of these problems and lets me actually watch them happen using metrics and logs.

intended users. two kinds of people would use this: an on call engineer who wants to glance at a dashboard and know if orders are backing up or requests are slow and a developer who is debugging one specific failed order and needs to find its exact log line fast.

the solution. quickbite exposes a simple order lifecycle:

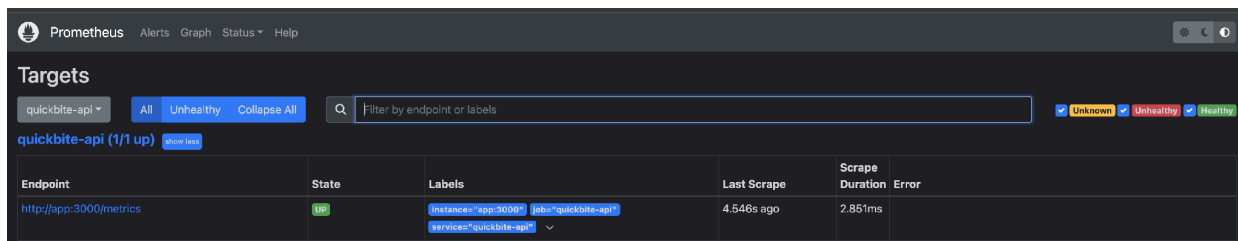
- POST /orders : place a new order
- POST /orders/:id/ready : mark an order ready (kitchen finished it)
- POST /orders/:id/cancel : cancel an order
- GET /orders and GET /orders/:id : view orders
- GET /health : health check
- GET /metrics : prometheus scrape endpoint
- POST /admin/fault : hidden switch used only for the part e experiment, to turn on artificial latency

it is a backend api on purpose and there is no frontend ui as in the assignment the assignment does not ask for a ui, so i interact with it using curl commands and small scripts, which is normal to use and test a backend service.

Just incase you wanna try it here is how to try it the full instructions are still in the readme file.

```
docker compose up --build
curl -X POST localhost:3000/orders -H "Content-Type: application/json" -d
'{"items":[{"name":"Zinger Burger","price":5.5,"qty":1}]}'
```

i tested every endpoint by hand with curl, including edge cases like ordering with no items (returns 400) and looking up an order id that does not exist (returns 404). the screenshot below shows the app is up and being scraped correctly by prometheus.



prometheus target for quickbite-api, state UP

part b: metrics

quickbite uses all four prometheus metric types across the project:

metric table

metric name	type	purpose	labels	where recorded
orders_total	counter	business metric , running count of orders by outcome	status (placed/cancelled)	server.js, POST /orders and POST /orders/:id/cancel
orders_pending	gauge	business metric , orders currently waiting on the kitchen	none	incremented in POST /orders, decremented in .../ready and .../cancel
order_processing_seconds	summary	business metric , time from placed to ready, avg + quantiles	none	POST /orders/:id/ready
http_request_duration_seconds	histogram	application metric , request latency, used for p95/p99	method, route, status code	logging middleware, runs on every request
http_requests_total	counter	application metric , total requests, used for failed-request rate	method, route, status code	same middleware
demo_requests_total	counter	used only for the part E.2 cardinality experiment	request_id (labeled mode only)	experiments/cardinality-app/server.js
node exporter metrics	counter/gauge	host resource usage (cpu, memory, disk, network)	instance, mode, mountpoint, device	node exporter, machine="quickbite-dev-host"

why a summary and not a histogram for kitchen time

order_processing_seconds is a summary because it is a low volume event meaning one sample per order, not per http request so having the client calculate the quantiles itself is fine here. i used node's prom-client library, which is actually able to calculate percentiles on the client side for a summary. the assignment specifically points out that python's prometheus_client summary type cannot do this and can only report count/sum/average, so if this project had been written in python i would have had to use a histogram with histogram_quantile() for the summary's p95/p99 instead, exactly like i already did for http_request_duration_seconds.

p95 / p99 and what the time window means

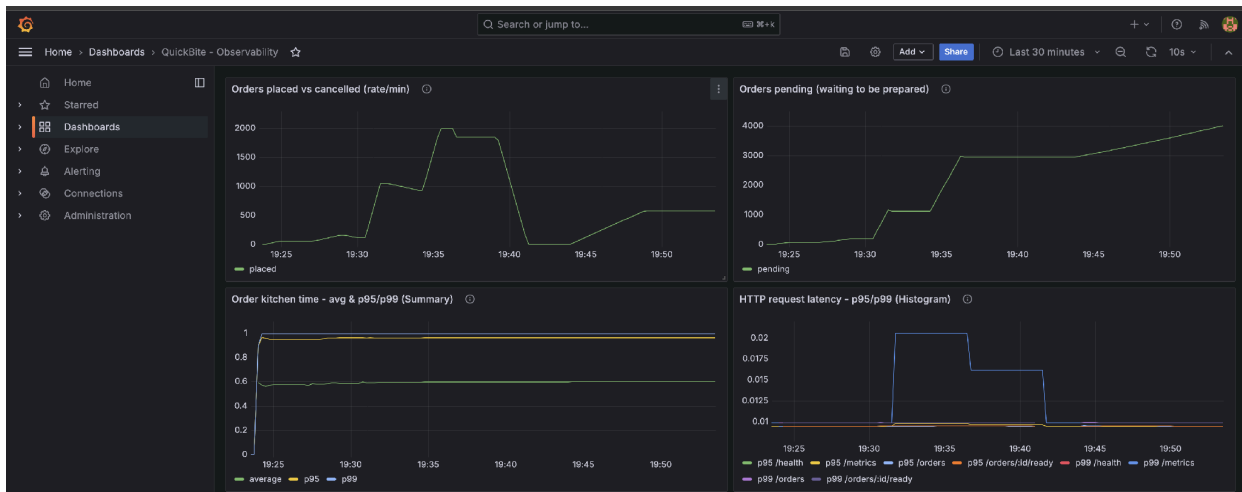
the main latency panel query is:

```
histogram_quantile(0.95, sum by (le, route) (rate(http_request_duration_seconds_bucket[5m])))
```

the [5m] part means the rate is calculated using the trailing 5 minutes of data, recalculated on every scrape (every 5 seconds, based on our scrape_interval). so this p95 line answers that out of requests in the last 5 minutes, 95% of them were this fast or faster. it is a rolling window, not an all-time total, which is why the line reacts with a small lag when the fault gets turned on and off later in part e.

grafana dashboard

the grafana dashboard is called quickbite-observability and is auto loaded when the stack starts, no manual setup needed. it has 9 panels covering all the metric types plus host metrics.



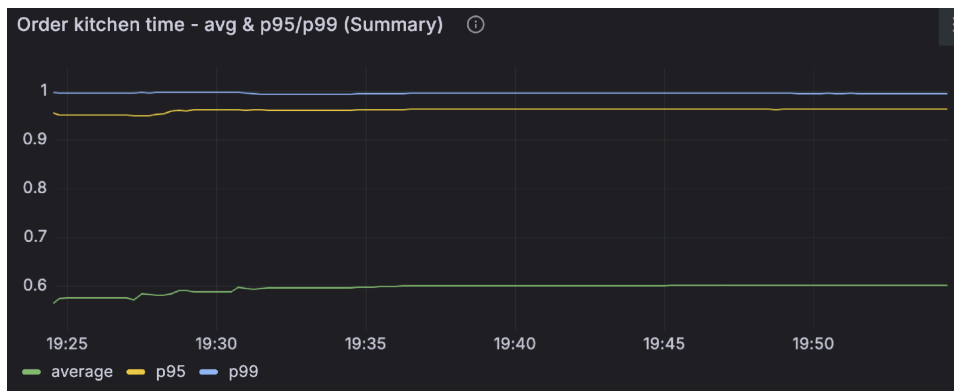
grafana dashboard, top half: orders placed vs cancelled, orders pending, kitchen time summary, http latency histogram



grafana dashboard, bottom half: kitchen time and latency again, failed requests, cardinality panel, and host cpu/memory/disk+network

the "failed requests (5xx) rate" panel shows no data in the screenshot above, which is actually a good sign, it means during that time window the app did not return a single server error. the active prometheus time series (cardinality demo) panel also shows no data here because the cardinality experiment (part e.2) had not been run yet at that point, that panel gets covered separately in part e.2 below.

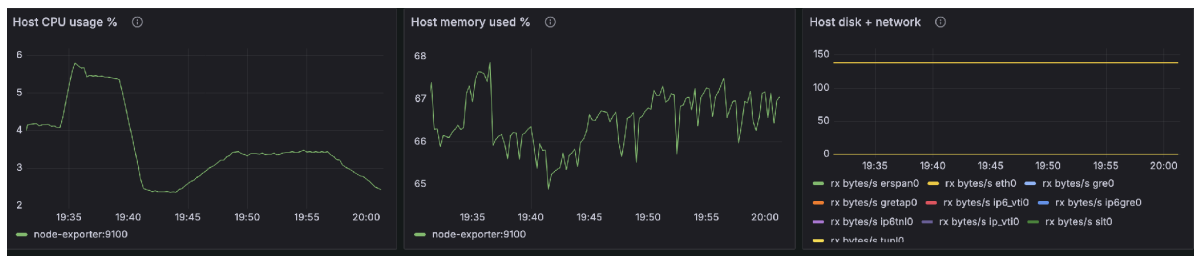
close up of the summary panel (order kitchen time, average + p95 + p99):



close up of order kitchen time summary panel

kitchen time here is simulated by the load generator script, which waits a random amount of time (200ms to 1000ms) after placing an order before marking it ready, standing in for real kitchen prep time since this app has no actual food being cooked. the panel shows average around 0.6s and p95/p99 close to 1s, which matches that random range.

close up of host metrics from node exporter (cpu / memory / disk + network):



close up of host cpu, memory, disk and network panels

the machine being measured is the docker host running docker compose up, labeled machine="quickbite-dev-host" in prometheus/prometheus.yml.

raw /metrics endpoint, confirming the endpoint itself works and returns real prometheus text format, this particular screenshot shows the default node.js process metrics near the top of the output, the custom business metrics like orders_total and orders_pending appear further down in the same file and are what feed the dashboard panels above:

```

macenergy@Interlinks-MacBook-Pro quickbite-observability 2 % curl localhost:3000/metrics
# HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE process_cpu_user_seconds_total counter
process_cpu_user_seconds_total 79.550314

# HELP process_cpu_system_seconds_total Total system CPU time spent in seconds.
# TYPE process_cpu_system_seconds_total counter
process_cpu_system_seconds_total 36.34649599999998

# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 115.89681000000009

# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1789913708

# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 82788352

# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 740003840

# HELP process_heap_bytes Process heap size in bytes.
# TYPE process_heap_bytes gauge
process_heap_bytes 110354432

# HELP process_open_fds Number of open file descriptors.
# TYPE process_open_fds gauge
process_open_fds 22

# HELP process_max_fds Maximum number of open file descriptors.
# TYPE process_max_fds gauge
process_max_fds 1048576

# HELP nodejs_eventloop_lag_seconds Lag of event loop in seconds.
# TYPE nodejs_eventloop_lag_seconds gauge
nodejs_eventloop_lag_seconds 0.006309991

# HELP nodejs_eventloop_lag_min_seconds The minimum recorded event loop delay.
# TYPE nodejs_eventloop_lag_min_seconds gauge
nodejs_eventloop_lag_min_seconds 0.010141696

# HELP nodejs_eventloop_lag_max_seconds The maximum recorded event loop delay.
# TYPE nodejs_eventloop_lag_max_seconds gauge
nodejs_eventloop_lag_max_seconds 0.011747327

# HELP nodejs_eventloop_lag_mean_seconds The mean of the recorded event loop delays.
# TYPE nodejs_eventloop_lag_mean_seconds gauge
nodejs_eventloop_lag_mean_seconds 0.010440586129496403

# HELP nodejs_eventloop_lag_stddev_seconds The standard deviation of the recorded event loop delays.
# TYPE nodejs_eventloop_lag_stddev_seconds gauge
nodejs_eventloop_lag_stddev_seconds 0.00037099575763427633

```

raw output of curl localhost:3000/metrics

metric i explored myself. the active prometheus time series (cardinality demo) panel is not part of the standard food ordering example from the assignment brief, i added it specifically so the part e.2 cardinality experiment would be visible directly on the dashboard instead of only in a terminal. it is covered with real results in part e.2 below.

part c: logs

what is logged, why, and where, (screenshot of logs attached below i picked the top one to explain with)

every http request produces one "request completed" log line with method, route, status code and duration, written by the logging middleware in server.js. domain events get their own log lines too: "order placed", "order ready", "order cancelled", "order rejected: no items", and "fault injection: delaying request" (used in part e.1). every log line includes a requestId so all the logs from one http call can be tied together, plus non sensitive fields like orderId, itemCount, amount, and processingSeconds. no secrets or personal data are ever logged, customerRef is just a made up string like cust_42 and nothing like passwords or tokens appears anywhere in the code.

parsing

the app writes json log lines to /app/logs/quickbite.log inside a shared docker volume. then filebeat mounts that same volume read only and tails the file using a filestream input with the ndjson parser, which reads each line as json and promotes its keys (timestamp, level, message, requestId, etc) straight to top level fields. no logstash step was needed because the app already writes structured json. a timestamp processor maps the apps own timestamp field onto elasticsearch's @timestamp field so kibana's time picker lines up with when the event actually happened in the app.

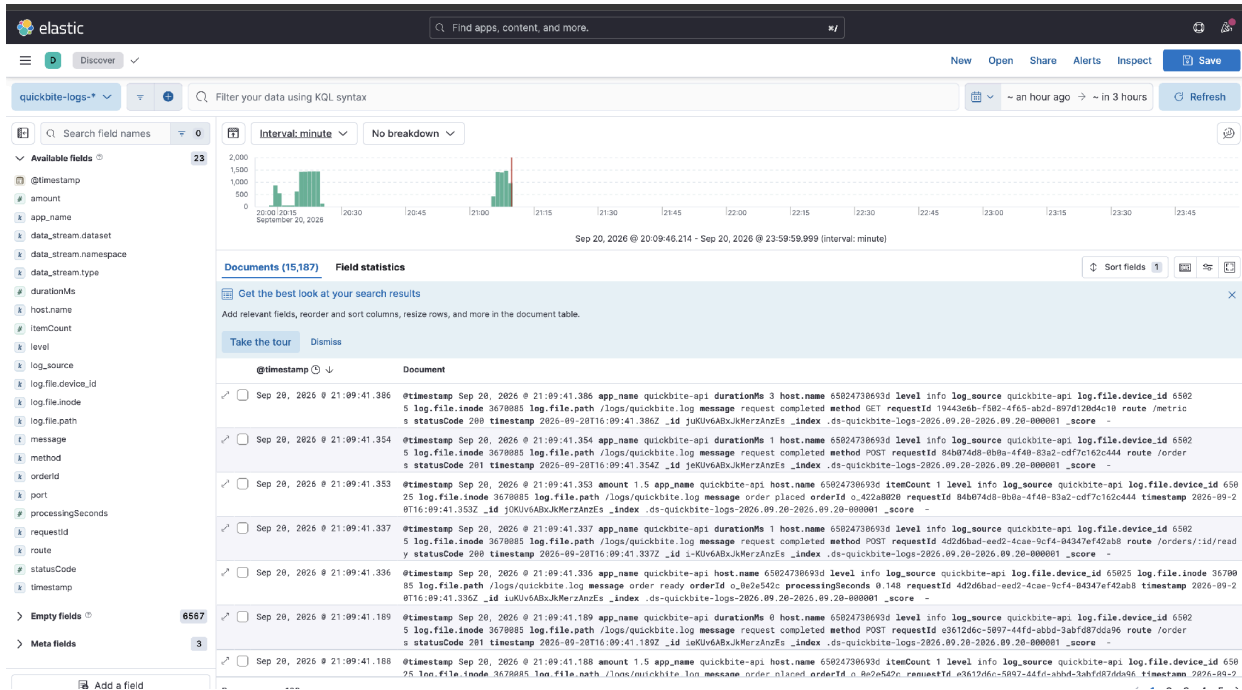
where logs live and when they are deleted

logs sit briefly in the app-logs docker volume as the raw json file, then get shipped into elasticsearch as indexed documents, one index per day (quickbite-logs-YYYY.MM.DD). old logs are deleted manually, for example curl -X DELETE

localhost:9200/quickbite-logs-2026.09.01. filebeat's own read position meaning how far into the file it has read, is saved in the filebeat-data volume, so if filebeat restarts it resumes instead of re-shipping everything.

confirming it in kibana

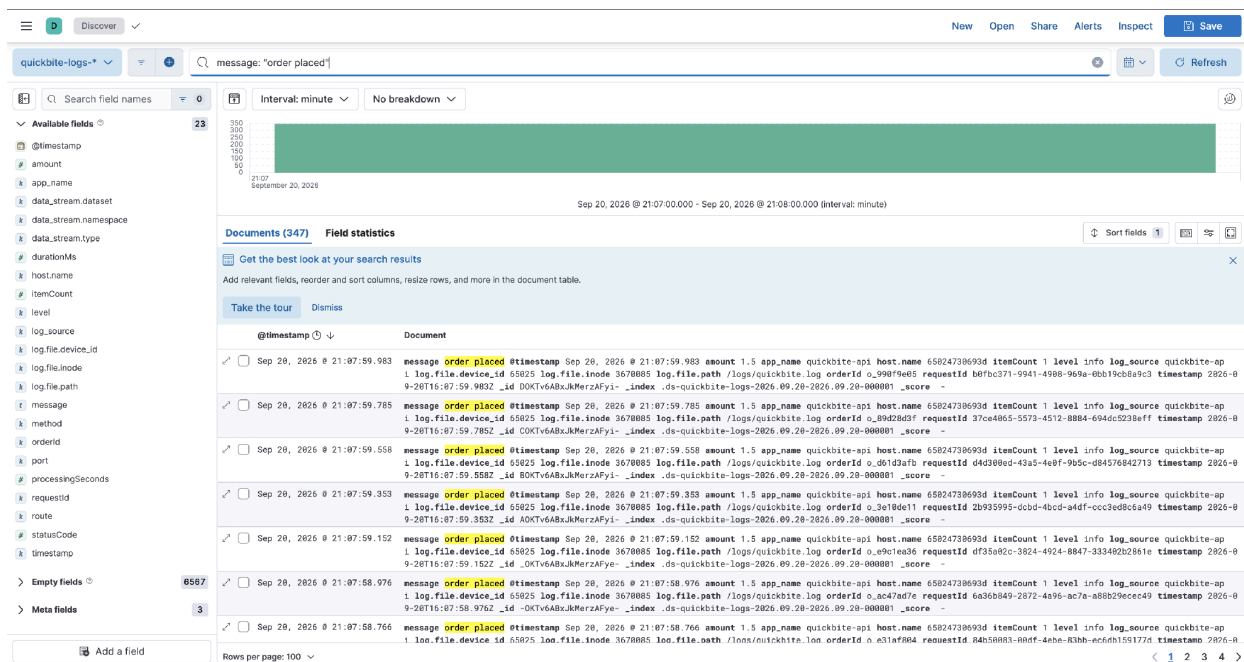
after picking the right data view, quickbite-logs-*, and widening the time range here then discover shows the real log stream, over 15,000 documents across the run:



kibana discover showing quickbite-logs-* documents and one expanded document with fields like app_name, requestId, orderId, message

each field from the original json line becomes its own searchable field in kibana: timestamp, level, message, app_name, requestId, orderId, amount, itemCount, route, method, statusCode, durationMs, log_source.

a working search. searching message: "order placed" returns just the matching documents, 347 in this run, with the search term highlighted:



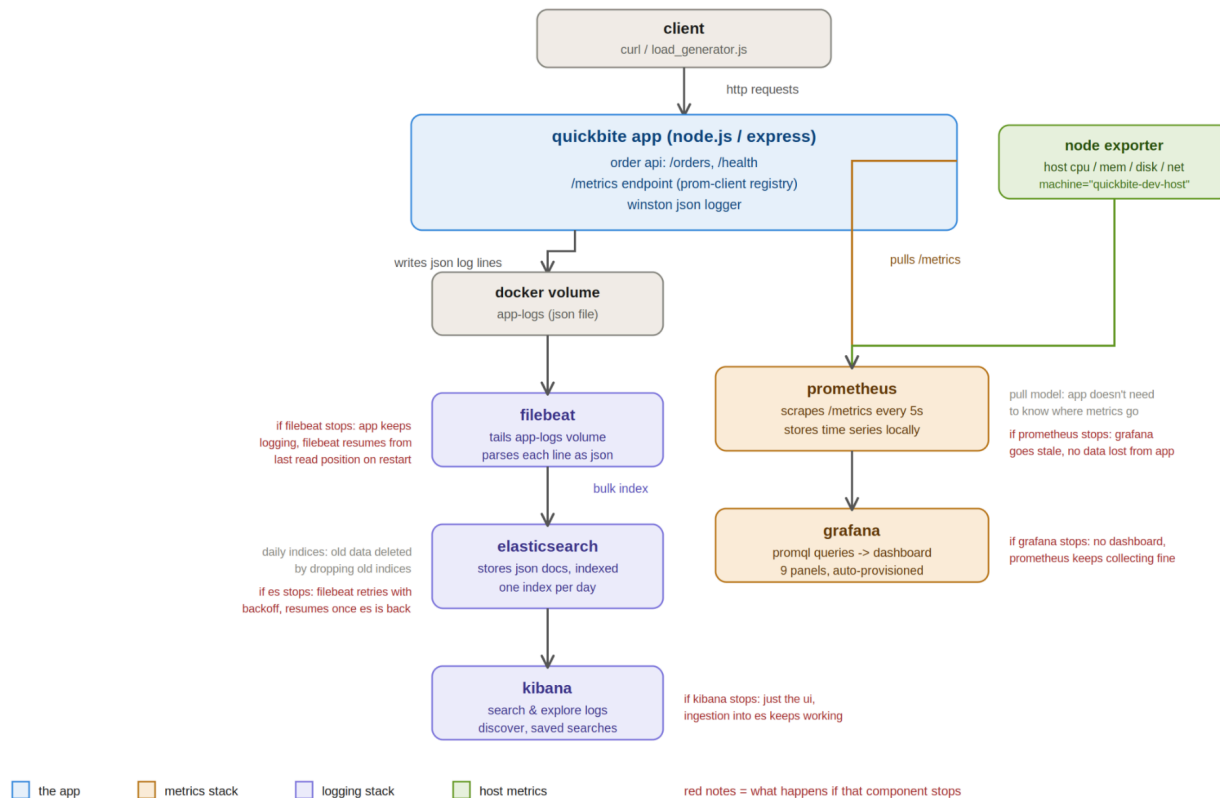
kibana search for message: order placed, 347 results, highlighted

part d: system design

architecture diagram

quickbite architecture

app, dependencies, and monitoring tools, with data flow and storage



- client sends http requests (curl or load_generator.js) to the quickbite app
- the app updates prometheus metrics in-process and writes a json log line via a logging middleware
- logs are written to a shared docker volume (app-logs)
- filebeat tails that volume, parses the json, and ships it to elasticsearch
- kibana lets you search and explore what elasticsearch stored
- prometheus scrapes the app's /metrics endpoint and node exporter every 5 seconds
- grafana queries prometheus with promql and draws the dashboard

what each part does here is that the quickbite app handles the order lifecycle and updates prometheus metrics in process, plus writes one json log line per event. prometheus pulls (scrapes) /metrics from the app and node exporter every 5 seconds and stores it in its own database. node exporter is a separate small process reporting host level cpu/memory/disk/network. grafana queries prometheus with promql and draws the dashboard panels. filebeat tails the shared log file, parses it as json, and ships it to elasticsearch, which stores it in daily indices. kibana lets you search and explore that stored data.

The metrics live in prometheus's own local database, i picked prometheus's pull model because it decouples the app from having to know where to send metrics, prometheus just needs to reach app:3000/metrics. logs live in elasticsearch as indexed json documents, one index per day, which makes it simple to delete old data by dropping old daily indices.

i chose the top down design since that's the real order requests flow in so i split metrics and logs into two side by side branches because they don't depend on each other. And colors group components by category so it's clear what belongs to which stack. arrows are labeled with the actual action happening, like pulls /metrics so the diagram shows how things communicate and storage notes sit only next to prometheus and elasticsearch since those are the only two things that actually store data long term. also each box has its own small red note on what happens if that part goes down, right on the diagram as since that's what the assignment asks for.

what happens if something stops working

component stops	what happens
the app	prometheus scrapes start failing (target goes down), no new logs get written, but nothing that already exists is lost
prometheus	grafana panels stop getting new points, the app keeps running fine, but any metrics generated while prometheus is down are gone since it is a pull model with no buffering
grafana	no dashboard, but prometheus keeps collecting and storing normally in the background
node exporter	only the host metrics panels go stale, business/app metrics are unaffected since they come from a different target
filebeat	the app keeps writing to its log file, nothing is lost as long as disk space holds up. when filebeat comes back, it remembers its last read position and resumes from there
elasticsearch	filebeat retries sending with its own backoff, and since it remembers the file offset, nothing is lost once elasticsearch is back, as long as the log file itself is not rotated away in the meantime
kibana	just the ui goes down, the actual pipeline from filebeat into elasticsearch keeps working fine

following one metric

orders_pending (a gauge): the code calls .inc() in POST /orders and .dec() in .../ready and .../cancel. prometheus scrapes app:3000/metrics every 5 seconds and stores the current value each time. grafana just plots that value directly with no extra math needed since it is already a gauge.

following one log

an "order placed" event: the code writes a json line to /app/logs/quickbite.log. filebeat tails that file, decodes the json with its ndjson parser, and maps the app's own timestamp field to @timestamp. elasticsearch indexes it into that day's index. kibana's discover, searching message: "order placed", finds it with all its fields available. the content of the message does not change along this path, but the envelope around it does, filebeat adds a log_source field from a static config and drops a few noisy fields (agent, ecs, input, log.offset) that were not needed.

part e: experiments

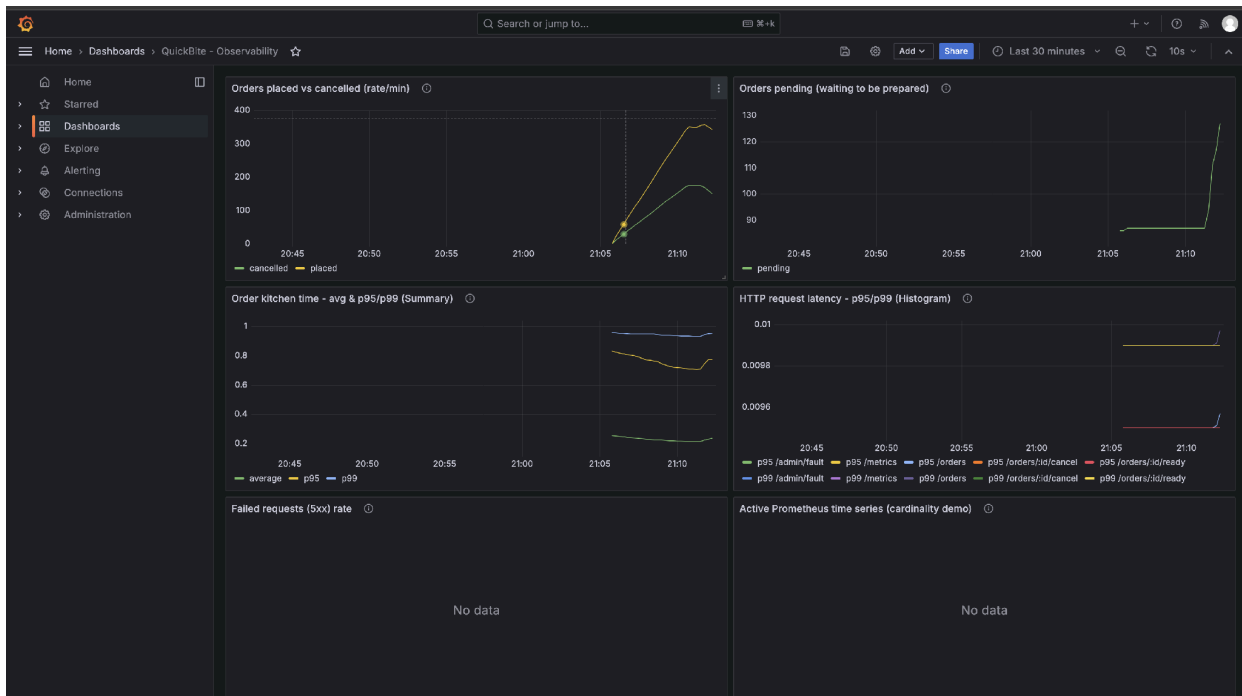
e.1: reproduce a problem (fault injection)

for this i picked a slow downstream dependency as the problem, the kind of thing that happens in real apps when e.g. a payment gateway or another service starts responding slowly, even though nothing is actually broken or returning errors.

server.js has a hidden switch, POST /admin/fault, that when turned on adds a 500ms delay to every 5th POST /orders request. experiments/fault_experiment.sh runs this as a 3 stage repeatable test, 30 seconds each at 5 requests per second:

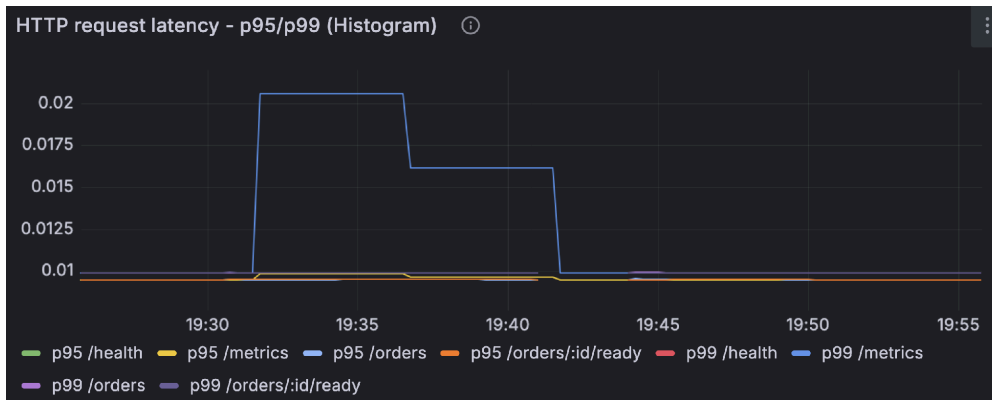
- before : fault off, this is the healthy baseline
- during : fault on, every 5th order gets delayed 500ms
- after : fault off again, to check the app recovers

So before the baseline latency is tiny like under 0.01s, and the panel is basically flat and ive added a screenshot below with a tiny spike



grafana dashboard during the before stage, http latency panel flat and tiny

during and after, close up of the http latency histogram panel which is showing the spike while the fault is on and then the drop back down once it is turned off, the timestamps are off as ive tested this multiple times and taken screenshots



close up of http request latency histogram panel showing a clear spike then recovery

terminal output at the end of the script, showing the after stage finished with all requests succeeding and the suggested promql query for this writeup:

```
Done. sent=147 ok=147 failed=0 avgMs=4.0
Screenshot the same panel for this window as AFTER - latency should
drop back to baseline, confirming recovery.

Experiment complete. Suggested Grafana query for the write-up:
  histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket{route="orders"}[1m])))
❖ macenergy@Interlinks-MacBook-Pro quickbite-observability 2 %
```

terminal showing fault_experiment.sh completing, sent=147 ok=147 failed=0, and the suggested grafana query

checking the actual warning logs fired, both at the source and inside elasticsearch:

```
macenergy@Interlinks-MacBook-Pro quickbite-observability 2 % docker compose exec app grep "fault injection" /app/logs/quickbite.log
{"app_name":"quickbite-api","delayMs":500,"level":"warn","message":"fault injection: delaying request","requestId":"e832d
db8-c1d4-461b-8f5b-2e7b64398e84","timestamp":"2026-09-20T16:12:09.301Z"}
{"app_name":"quickbite-api","delayMs":500,"level":"warn","message":"fault injection: delaying request","requestId":"88211
4b7-2e38-4bd6-a57b-50f367f80ac2","timestamp":"2026-09-20T16:12:10.821Z"}
{"app_name":"quickbite-api","delayMs":500,"level":"warn","message":"fault injection: delaying request","requestId":"8d79d
5e2-15e7-4f0a-b2ae-44f44c914a77","timestamp":"2026-09-20T16:12:12.336Z"}
{"app_name":"quickbite-api","delayMs":500,"level":"warn","message":"fault injection: delaying request","requestId":"d4c65
a49-1a7f-426c-8e9a-dd27a6cc3a91","timestamp":"2026-09-20T16:12:13.858Z"}
{"app_name":"quickbite-api","delayMs":500,"level":"warn","message":"fault injection: delaying request","requestId":"d1699
ae4-7577-4f5e-b42a-6f5b1d7dfa5","timestamp":"2026-09-20T16:12:15.378Z"}
{"app_name":"quickbite-api","delayMs":500,"level":"warn","message":"fault injection: delaying request","requestId":"5d353
4cd-25af-4dc2-965a-09a2c6847894","timestamp":"2026-09-20T16:12:16.896Z"}
{"app_name":"quickbite-api","delayMs":500,"level":"warn","message":"fault injection: delaying request","requestId":"44782
80f-d033-49ca-8ae2-0808e3c575b","timestamp":"2026-09-20T16:12:18.417Z"}
{"app_name":"quickbite-api","delayMs":500,"level":"warn","message":"fault injection: delaying request","requestId":"db357
e18-298c-439c-80c9-d0f53a9a7394","timestamp":"2026-09-20T16:12:19.935Z"}
{"app_name":"quickbite-api","delayMs":500,"level":"warn","message":"fault injection: delaying request","requestId":"a44fa
4ee-8e97-48bb-a21c-8ecbbafc14ea","timestamp":"2026-09-20T16:12:21.454Z"}
{"app_name":"quickbite-api","delayMs":500,"level":"warn","message":"fault injection: delaying request","requestId":"00c90
017-8820-4a2f-ad1e-fdbf728a3e07","timestamp":"2026-09-20T16:12:22.973Z"}
macenergy@Interlinks-MacBook-Pro quickbite-observability 2 % curl -s "localhost:9200/quickbite-logs-*/_search?q=message:%
22fault%20injection%22" | node -pe 'JSON.parse(require('fs').readFileSync(0,'utf8')).hits.total'
{ value: 20, relation: 'eq' }
macenergy@Interlinks-MacBook-Pro quickbite-observability 2 %
```

terminal showing grep of fault injection warning logs from the app, and a curl to elasticsearch confirming 20 matching documents were indexed

cause and effect on users is that a slow downstream dependency, simulated here but standing in for something like a slow payment gateway or kitchen display system in a real app it makes customers wait longer to get an order confirmation without the app returning any errors at all so this is exactly why latency metrics matter and not just error rate metrics as an error rate graph alone would have shown nothing wrong here even though 1 in 5 customers were having a bad experience.

So my result summary is that :

20 "fault injection: delaying request" warning log lines were confirmed both in the app's own log file and indexed into elasticsearch. the http latency histogram panel shows a clear step up during the fault window and a clear step back down once it was turned off, confirming recovery. the full script run completed with 147 requests sent, 147 successful, 0 failed, meaning the fault only adds latency, it never causes actual request failures which matches what i predicted.

e.2: cardinality explosion

for this i used a separate isolated demo app and its in the folder only as experiments/cardinality-app/server.js. so the experiment could not mess with the real business dashboards. it exposes one prometheus counter, demo_requests_total, and can run in two modes controlled by an environment variable:

- LABELED=true : every request adds a unique request_id label value
- LABELED=false : no label at all, just a single counter going up

cardinality-demo (1/1 up) Show logs					
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://cardinality-demo:9091/metrics	UP	instance="cardinality-demo-9091" x k8s="cardinality-demo" v	478.000ms ago	3.313ms	
node-exporter (1/1 up) Show logs					

prometheus target cardinality-demo, state UP, correctly resolved by service name

step 1, labeled mode. sent 100 requests, each with a different request_id, waited for two prometheus scrapes, then queried count(demo_requests_total):

```
macenergy@Interlinks-MacBook-Pro quickbite-observability 2 % for i in $(seq 1 100); do curl -s localhost:9091/hit > /dev
null; done
sleep 12
curl -s "http://localhost:9090/api/v1/query" --data-urlencode 'query=count(demo_requests_total)' | node -pe 'JSON.parse(
require('fs').readFileSync(0,'utf8')).data.result[0].value[1]'
100
```

terminal showing 100 requests sent in labeled mode and count query returning 100

result: 100 - one time series was created per unique request id.

step 2, unlabeled mode. This stopped the labeled container, started the same app again with LABELED=false, sent another 100 requests, and queried the same count:

```

✓ Image quickbite-observability2-cardinality-demo Built 15.
✓ Container quickbite-cardinality-demo Started
macenergy@Interlinks-MacBook-Pro quickbite-observability 2 % for i in $(seq 1 100); do curl -s localhost:9091/hit > /dev
null; done
sleep 12
curl -s "http://localhost:9090/api/v1/query" --data-urlencode 'query=count(demo_requests_total)' | node -pe "JSON.parse(
require('fs').readFileSync(0,'utf8')).data.result[0].value[1]"
1
macenergy@Interlinks-MacBook-Pro quickbite-observability 2 %

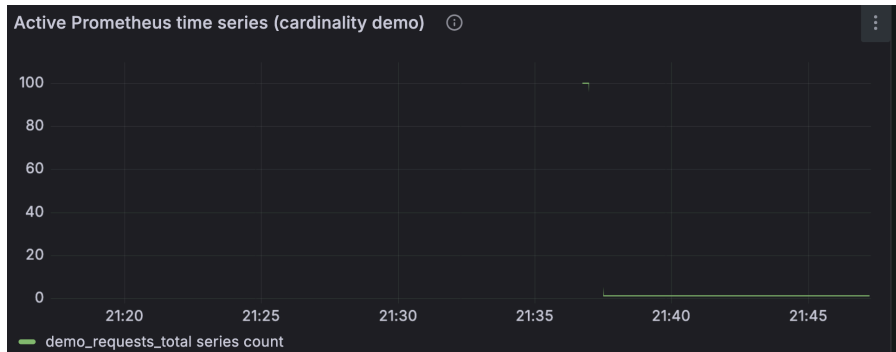
```

terminal showing 100 requests sent in unlabeled mode and count query returning 1

result: 1 - a single series that just had its internal count go up to 100, instead of 100 separate series.

visually on the dashboard.

the "active prometheus time series (cardinality demo)" panel shows the step up to 100 during the labeled run, then it drops back down to a flat near zero line once switched to the unlabeled app, since that mode only ever produces 1 series:



grafana panel showing a step up to 100 series then a drop back to near zero

At a larger scale the cost is that each unique label value combination becomes a completely separate time series in prometheus's internal database, with its own memory and disk footprint. a label with unbounded cardinality, like request ids, user ids, raw urls, or ip addresses, effectively turns one counter into unlimited counters. at real production traffic volumes this is exactly how teams accidentally take down their own prometheus instance through memory exhaustion or extremely slow queries. this is why high cardinality values like request ids belong in logs, where elasticsearch handles high cardinality fields just fine, and never in metric labels, where prometheus does not.

stopping the labeled app and switching to the unlabeled one stops new samples being written under the old high cardinality series, but it does not delete the series that were already stored, they just sit there until they age out based on prometheus's normal retention settings.

credits

built using express, prom-client, and winston for the app, prometheus, grafana and node exporter for metrics, and elasticsearch, kibana and filebeat for logs, all standard open source tools as suggested by the assignment. all endpoints, metrics, log fields, the dashboard, and the experiment scripts were written specifically for this project, nothing was copied from another source. i did use claude to help me build and debug this project however all screenshots, understanding, metrics etc seen, dashboards, were setup by me.